॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 05

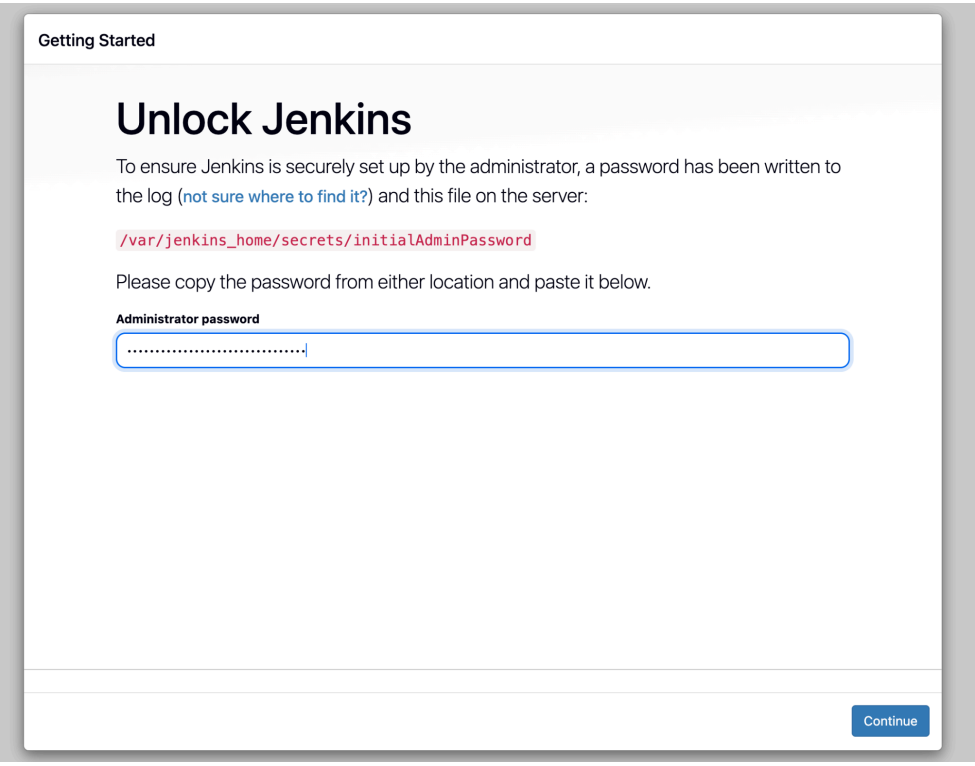
Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Yashmit Kotekar**
PRN No.	**23070122236**
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	16th July, 2026
Title of Assignment:	Continuous Integration with Jenkins
Theory:	**Theory** Jenkins is an open-source automation server used to implement Continuous Integration (CI) and Continuous Delivery (CD). It automates software development tasks such as building, testing, and deploying applications. Jenkins integrates with version control systems like Git to automatically retrieve source code and execute predefined build steps. In this experiment, Jenkins was installed using Docker, configured with the Git plugin, and connected to a GitHub repository. A Freestyle project was created to pull source code from the repository and execute shell commands. The build was then triggered manually, and the console output was examined to verify successful execution. **Commands Used** • docker --version – Verified that Docker was installed on the system. • docker ps – Checked whether the Jenkins Docker container was running. • docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home --name myjenkins jenkins/jenkins:lts-jdk17 – Created and started the Jenkins Docker container. • docker logs myjenkins – Displayed the Jenkins logs to obtain the initial

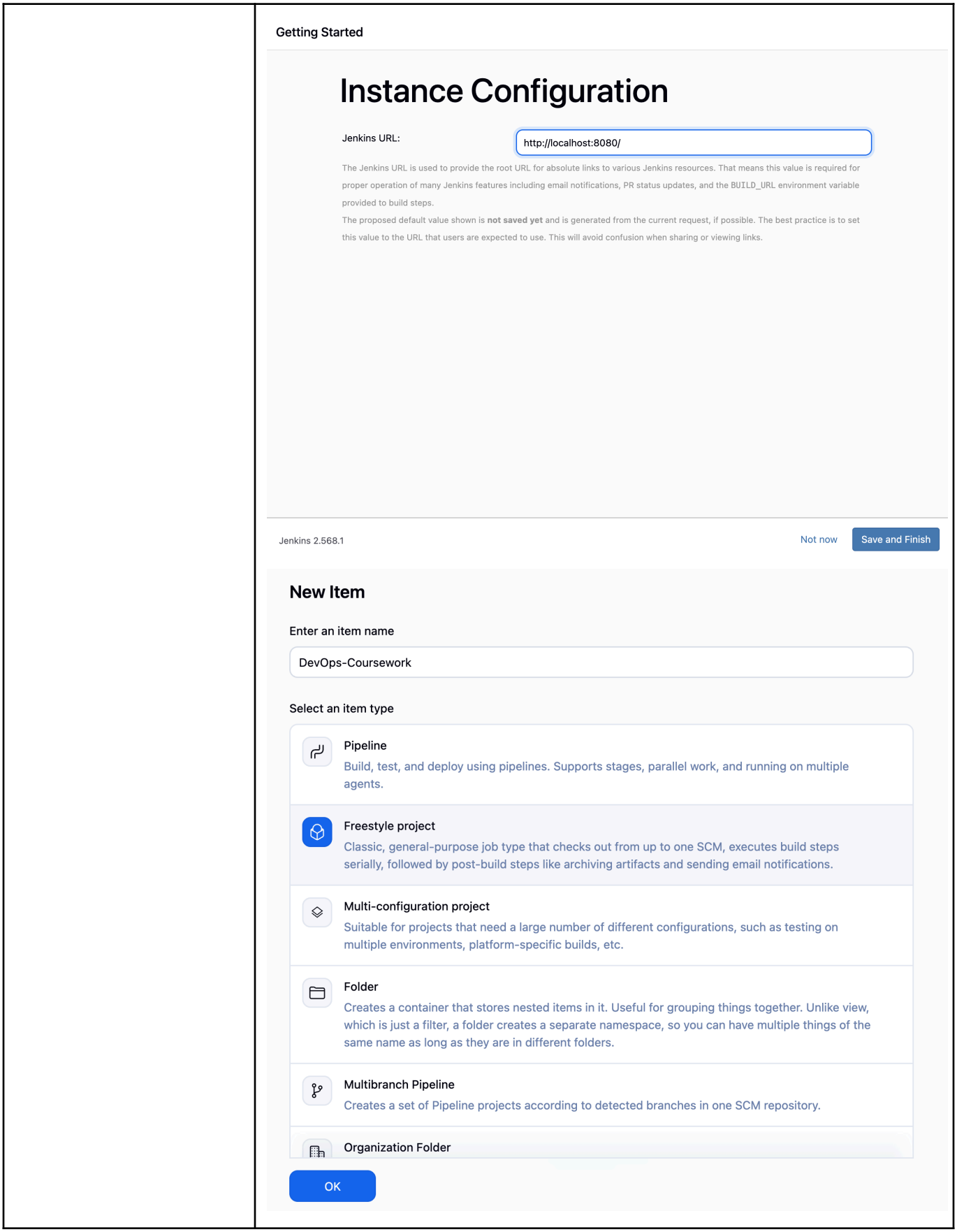
	administrator password during the initial setup. ● docker exec myjenkins cat /var/jenkins_home/secrets/initialAdminPassword – Retrieved the initial administrator password generated by Jenkins. **Execute Shell Commands** ● echo "BUILDING: ${JOB_NAME}" – Displayed the Jenkins job name. ● echo "BUILD NUMBER: ${BUILD_NUMBER}" – Displayed the current build number. ● echo "WORKSPACE: ${WORKSPACE}" – Displayed the Jenkins workspace path. ● ls -la – Listed all files present in the Jenkins workspace. ● echo "Hello Jenkins!" – Displayed a sample message to verify successful execution of the build step.
Implementation:	**Step 1: Verify Docker and Jenkins Container** Verified Docker and Jenkins Container — Verified Docker installation and confirmed that the Jenkins Docker container (myjenkins) was running successfully. **Step 2: Access Jenkins Dashboard** Accessed Jenkins Dashboard — Successfully accessed the Jenkins Dashboard and verified that the existing Freestyle project (Git-Practice-Build) was available. **Step 3: Verify Git Plugin Installation** Verified Git Plugin — Verified that the Git plugin was installed and enabled, allowing Jenkins to integrate with Git repositories for source code management. **Step 4: Open the Freestyle Project** Opened Freestyle Project — Opened the existing Freestyle project (Git-Practice-Build) to verify its configuration and prepare it for executing the build process. **Step 5: Verify Source Code Management (Git)** Configured Source Code Management — Configured the Freestyle project to retrieve source code from the GitHub repository by selecting Git as the Source Code Management option and specifying the repository URL. **Step 6: Verify Build Step** Configured Build Step — Verified the Execute shell build step containing shell commands to display build information, list the repository files, and print a sample message during the build process. **Step 7: Save and Run the Build**

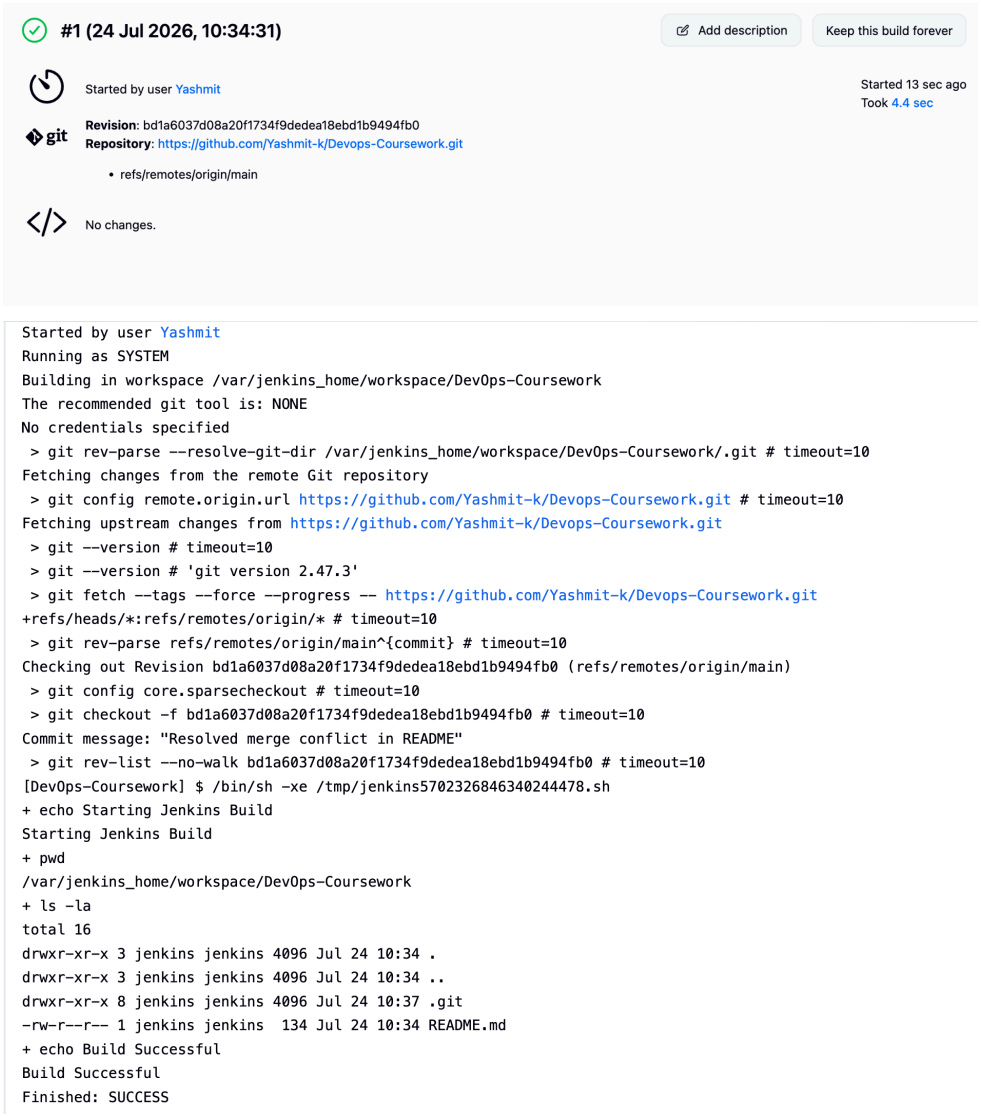
Executed Build — Successfully executed the Freestyle project using Build Now and verified that the build completed successfully with a SUCCESS status.

Step 8: View Console Output

Verified Console Output — Examined the console output and verified that Jenkins successfully cloned the Git repository, executed the configured shell commands, and completed the build with a SUCCESS status.




Getting Started
Instance Configuration
Jenkins URL:
http://localhost:8080/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.
Jenkins 2.568.1
Not now
Save and Finish
New Item
Enter an item name
DevOps-Coursework
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.
Organization Folder
OK



Subject Faculty Name: Prof. Arijit Dutta